

Discount Optimisation (discount)

Tommaso has just happened to know of a massive discount campaign in a supermarket in his neighbourhood. Being notoriously greedy, he is now planning to take the most advantage out of it!



Figure 1: Lots of discounts.

The supermarket is selling N unique items, each marked with a distinct identifier i from 0 to $N - 1$. Every item has a full price P_i , a discounted price S_i , and an attached promotional code R_i . Once a buyer has to pay for her shopping cart, she will qualify for the discounted price for a bought item i if and only if she has at least one *matching promotional code* attached to some item(s) in her shopping cart. Otherwise, she will have to pay the full price.

Notice that if item 0 gives a promotional code for item 1, and vice-versa item 1 provides a code for item 0, it is possible to buy items 0 and 1 both at their discounted prices. On the other hand, **it is impossible to buy multiple copies of the same item**, and having several promotional codes for a single item does not increase the discount. It is however possible for an item to provide a promotional code for itself, so that it is never paid full price.

Tommaso is willing to buy just about anything from the supermarket, in order to boast the highest possible *discount percentage*. Assume that Tommaso chooses to buy a certain subset of the items, whose total full price is P_{tot} , for which he pays an actual price of P_{discount} using promotional codes. Then the discount percentage for those items is computed as:

$$100 \left(1 - \frac{P_{\text{discount}}}{P_{\text{tot}}} \right)$$

Help Tommaso find the best possible bargain!

📎 Among the attachments of this task you may find a template file `discount.*` with a sample incomplete implementation.

Input

The first line contains the only integer N . The following N lines each contain the three integers P_i , S_i and R_i for item i .

Output

You need to write a single line with a floating-point number: the best percentage of discount Tommaso can obtain.

Constraints

- $1 \leq N \leq 100\,000$.
- $0 \leq S_i < P_i \leq 10\,000$ for each $i = 0 \dots N - 1$.
- $0 \leq R_i < N$ for each $i = 0 \dots N - 1$.

Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases. Your answer is considered correct if it is within 10^{-6} of the correct value.

- | | |
|---|---|
| – Subtask 1 (0 points) | Examples. |
|  | |
| – Subtask 2 (10 points) | $R_i = i$ for all i . |
|  | |
| – Subtask 3 (15 points) | $N \leq 20$. |
|  | |
| – Subtask 4 (20 points) | $N \leq 1000$. |
|  | |
| – Subtask 5 (20 points) | $R_i \leq i$ for all i . |
|  | |
| – Subtask 6 (25 points) | $R_0 = N - 1$ and $R_i = i - 1$ for all $i > 0$. |
|  | |
| – Subtask 7 (10 points) | No additional limitations. |
|  | |

Examples

input	output
6 100 90 1 10 9 2 90 20 5 100 80 2 40 30 3 100 10 3	80.0000000000
5 100 70 1 10 3 2 11 3 3 12 3 1 10 9 4	72.727272727

Explanation

In the **first sample case** the best solution is to buy items 1, 2 and 5 spending $P_1 + S_2 + S_5 = 40$ instead of $P_1 + P_2 + P_5 = 200$. The discount percentage is $100 \left(1 - \frac{40}{200}\right) = 80\%$. This is because item 1's promotional code enables the discount of item 2, and the code of item 2 enables the discount of item 5.

In the **second sample case** the best solution is to buy items 1, 2 and 3 spending $S_1 + S_2 + S_3 = 9$ instead of $P_1 + P_2 + P_3 = 33$. The discount percentage is $100 \left(1 - \frac{9}{33}\right) = 72.7272727\%$. The three items enable the discount for each other, therefore all of them are paid with the sale price.